

Installing the GoLearn package

Once Go is installed, you can install GoLearn using *go get*:

```
go get github.com/sjwhitworth/golearn
```

This command downloads and installs GoLearn and its dependencies. You can verify the installation by importing GoLearn in a Go script and running a simple test.

Key features of GoLearn

GoLearn simplifies ML tasks with modular, well-organised components. It manages datasets with instances, and supports CSV loading and format conversion. It enables feature scaling, normalisation, and data transformations; and implements decision trees, KNN, Naïve Bayes, and other algorithms.

GoLearn offers a range of powerful features that make it a great choice for machine learning in Go.

Data handling and preprocessing capabilities

GoLearn simplifies data handling through its base package, which provides structures to manage datasets efficiently. Here's how you can load data from a CSV file.

```
package main

import (
    "fmt"
    "github.com/sjwhitworth/golearn/base"
)

func main() {
    // Load a CSV dataset into GoLearn's Instances structure
    data, err := base.ParseCSVToInstances("dataset.csv",
    true)
    if err != nil {
        fmt.Println("Error loading data:", err)
        return
    }

    // Print dataset summary
    fmt.Println(data)
}
```

Explanation:

- *base.ParseCSVToInstances("dataset.csv", true)* loads the CSV file into a structured format. The second argument (true) indicates that the dataset has a header row.
- If an error occurs while loading, it is handled gracefully.

Preprocessing: Normalisation and feature scaling: Feature scaling is crucial to ensure that machine learning models perform optimally. GoLearn provides preprocessing utilities for this.

```
import (
    "github.com/sjwhitworth/golearn/filters"
```

```
)

// Normalize data using min-max scaling
normalizer := filters.NewMinMaxScaler("FeatureColumn")
normalizer.Fit(data) // Compute scaling parameters from data
normalizedData := normalizer.Transform(data) // Apply
transformation
```

Explanation:

- *NewMinMaxScaler("FeatureColumn")* creates a scaler for a specific feature column.
- *Fit(data)* computes min-max scaling parameters from the dataset.
- *Transform(data)* applies normalisation to the dataset.

Built-in algorithms for classification, regression, and clustering

GoLearn provides various machine learning models for classification, regression, and clustering. Classification with decision trees is done as follows:

```
import (
    "github.com/sjwhitworth/golearn/trees"
)

// Create a decision tree classifier
tree := trees.NewID3DecisionTree(0.6) // 0.6 is the decision
threshold
tree.Fit(trainingData) // Train the model
predictions, _ := tree.Predict(testData) // Make predictions
```

Explanation:

- *NewID3DecisionTree(0.6)* initialises a decision tree with a confidence threshold of 0.6.
- *Fit(trainingData)* trains the model using the provided dataset.
- *Predict(testData)* makes predictions on new data.

K-Nearest Neighbors (KNN) classification

```
import (
    "github.com/sjwhitworth/golearn/knn"
)

// Create a KNN classifier with k=3
knnClassifier := knn.NewKnnClassifier("euclidean", "linear", 3)
knnClassifier.Fit(trainingData)
predictions, _ := knnClassifier.Predict(testData)
```

Explanation:

- *"euclidean"* uses Euclidean distance for similarity measurement.
- *"linear"* uses linear search (other options include tree-